

PHASE 1

Machine Learning-Based Binary Image Classification

label = 5



label = 0



label = 4



label = 1



label = 9



label = 2



label = 1



label = 3



label = 1



label = 4



label = 3



label = 5



label = 3



label = 6



label = 1



label = 7



label = 2



label = 8



label = 6



label = 9



Submitted to:

Dr. Hossam

Eng. Ali

Submitted by :

Omar Adly 23P0244

Omar Osama 2300090

Ali Moataz 23P0280

Loaiy Mahmoud 23P0419

Abdelrahman Adel 23P0144

1. Introduction & Problem Definition

This project focuses on binary handwritten digit classification using the MNIST dataset. The objective is to design and evaluate multiple machine learning models from scratch to distinguish a selected digit class from all remaining digits.

The classification task is mathematically represented as:

$$f: X \rightarrow Y$$

Where X represents the extracted feature vector from an image, and $Y \in \{0,1\}$ represents the binary class label.

The project compares the performance of several machine learning algorithms, including Logistic Regression, Support Vector Machine (SVM), Naive Bayes, Decision Tree, and K-Nearest Neighbors (KNN). Model evaluation is based on Accuracy, Precision, Recall, and F1-score, with emphasis on achieving strong generalization and minimizing classification errors.

2. Preprocessing & Feature Engineering

2.1 Data Preparation

MNIST consists of 28×28 grayscale images. Pixel values are normalized to [0,1] using $X_{norm} = X/255$. For binary classification, the target digit is mapped to class 1 and all others to class 0. The dataset is split into 80% training and 20% validation sets.

2.2 Feature Extraction

Three feature representations were evaluated:

- **Flattening:** Each image is converted into a 784-dimensional raw pixel vector.
 - **PCA:** Dimensionality reduction by projection onto principal components:

$$X_{PCA} = XW$$

The top 50 components are retained to preserve variance while reducing complexity.

- **HOG:** Captures structural information using gradient magnitude and orientation:

$$|\nabla I| = \sqrt{G_x^2 + G_y^2} \quad \theta = \tan^{-1} \left(\frac{G_y}{G_x} \right)$$

HOG preserves edge and shape features, making it highly effective for digit recognition.

2.3 Feature Standardization

All extracted features are standardized to zero mean and unit variance:

$$X_{std} = \frac{X - \mu}{\sigma}$$

This improves numerical stability and optimization consistency across models.

3.1 Logistic Regression

Logistic Regression models the probability of the positive class using a linear transformation followed by a sigmoid activation:

$$z = Xw + b$$
$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

For numerical stability, z is clipped to the range $[-500, 500]$. The weights are initialized using a small Gaussian distribution, while the bias is initialized to zero.

Loss Function

The model minimizes Binary Cross-Entropy with L2 regularization:

$$J(w, b) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] + \frac{\lambda}{2N} \sum_{j=1}^d w_j^2$$

Where N is the number of samples, d the feature dimension, and $\lambda=0.01$. The bias term is excluded from regularization. A small constant ϵ is used in log computations for numerical stability.

Optimization Strategy

Parameters are optimized using Batch Gradient Descent:

$$\frac{\partial J}{\partial w} = \frac{1}{N} X^T (\hat{y} - y) + \frac{\lambda}{N} w \qquad \frac{\partial J}{\partial b} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y)$$

The parameters are then updated simultaneously using the learning rate $\alpha = 0.01$:

$$w = w - \alpha \frac{\partial J}{\partial w} \qquad b = b - \alpha \frac{\partial J}{\partial b}$$

With learning rate $\alpha=0.01$. Training runs for a maximum of 1000 epochs with early stopping when loss improvement falls below 10^{-7} .

Final Model Selection

The HOG + Logistic Regression (L2) model with a decision threshold of 0.6 was selected as the final solution.

Compared to PCA and raw pixel flattening, HOG features captured structural image gradients more effectively, achieving the best overall performance with:

- Accuracy: 99.11%
- F1-Score: 0.9609

Threshold tuning showed that 0.6 provided the best balance between Precision and Recall. It reduced False Positives from 172 to 85 while maintaining strong detection capability for positive samples.

3.2 Support Vector Machine (SVM)

Support Vector Machines were implemented for binary classification using Linear and Polynomial Kernel variants. The Linear SVM optimizes:

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \max(0, 1 - y_i(w \cdot x_i + b))$$

where x_i is the input feature vector, $y_i \in \{-1, +1\}$ is the class label, w is the weight vector, b is the bias, and C controls the trade-off between margin maximization and classification error.

The decision function is defined as:

$$f(x) = w \cdot x + b$$

A sample is classified as:

- $y = 1$ if $f(x) \geq 0$
- $y = -1$ if $f(x) < 0$

Kernel Methods & Optimization

To handle non-linear distributions, the team implemented a Polynomial Kernel:

$$K(x_i, x_j) = (x_i \cdot x_j + c)^d$$

Where c is a constant and d is the polynomial degree.

To reduce computational complexity, the Nystroem Approximation was applied by approximating the kernel matrix using landmark points:

$$K(X, L) \in \mathbb{R}^{n \times m} \quad m \ll n$$

This reduces memory and computation cost while preserving kernel performance.

Model Selection

The Linear SVM + HOG model was selected as optimal based on performance and generalization. It achieved the highest recall (0.9682) with the lowest False Negatives (42), making it most effective for detecting the target digit.

HOG consistently outperformed PCA by preserving structural edge information, while PCA lost discriminative spatial details. Although Kernel SVM achieved lower training loss, Linear SVM showed better validation stability and generalization.

Overall, evaluation metrics (Recall and F1-score) were prioritized over training loss for final model selection.

3.3 Naive Bayes

Naive Bayes is a probabilistic classifier based on Bayes' Theorem. For an input feature vector $x=(x_1,x_2,...,x_d)$, the posterior probability is:

$$p(y|x) = \frac{p(y) \prod_{i=1}^n P(x_i|y)}{p(x)}$$

Since features are continuous (pixel intensities), a Gaussian likelihood is assumed:

$$p(x_i|y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} e^{-\frac{(x_i-\mu_y)^2}{2\sigma_y^2}}$$

To avoid numerical underflow, computations are performed in log-space:

$$\ln(p(y|x)) \propto \ln(p(y)) + \sum_{i=1}^n \ln(p(x_i|y))$$

Prediction is made using the Maximum A Posteriori (MAP) rule.

Parameter Estimation

Naive Bayes does not use iterative optimization. Parameters are estimated using Maximum Likelihood Estimation:

$$\mu = \frac{1}{N} \sum x_i \qquad \sigma^2 = \frac{1}{N} \sum (x_i - \mu)^2$$

Each class is modeled independently per feature.

Implementation Enhancements

- PCA (50 components) improves performance by reducing feature correlation
- Variance smoothing (10^{-4}) prevents numerical instability
- 5-fold cross-validation ensures stable performance across splits

Model Selection

The PCA + Naive Bayes configuration was selected as optimal due to improved stability and performance over raw features. PCA enhances the independence assumption, leading to better likelihood estimation. Although simpler models converge faster, PCA-based features provide more consistent classification performance across folds.

3.4 K-Nearest Neighbors (KNN)

K-Nearest Neighbors is a non-parametric, instance-based classifier that assigns a label based on the majority class among the k closest training samples.

The Euclidean distance metric is used:

$$d(x, x_i) = \sqrt{\sum_{j=1}^d (x_j - x_{i,j})^2}$$

Where x is the query sample, x_i is a training sample, and d is the feature dimension.

The algorithm has no training phase; all computations occur at prediction time by selecting the k nearest neighbors and applying majority voting.

Implementation Details

- Distance computation is fully vectorized using NumPy
- Neighbor selection uses `np.argpartition` for efficiency
- Batch prediction is used for scalability

- PCA is applied to reduce dimensionality from 784 to 50 features

KNN is implemented entirely from scratch without machine learning libraries.

Hyperparameter Tuning

The main hyperparameter is k , tested as follows:

k	Performance
1	Best
3	Moderate
5	Lower

The best performance was achieved at $k=1$, as it preserves local structure without smoothing class boundaries.

Results

Final evaluation (KNN, $k=1$, PCA 784 $\rightarrow$ 50):

- Accuracy = 0.9949
- Precision = 0.9617
- Recall = 0.9947
- F1-score = 0.9779

The confusion matrix shows very low misclassification, with only 45 false positives and 6 false negatives, indicating strong classification reliability.

Model Selection

KNN with $k=1$ and PCA reduction was selected as optimal due to its highest accuracy and recall. PCA improves efficiency by reducing dimensionality while preserving variance, and $k=1$ provides the most precise local decision boundaries. Overall, the model achieves strong performance with minimal false negatives and high classification reliability.

3.5 Decision Tree

A Decision Tree classifies an input $\mathbf{x} \in \mathbb{R}^{784}$ by recursively splitting the feature space into binary decisions until reaching a leaf node.

At each node, a split is defined by a feature j and threshold τ :

$$x_j \leq \tau \rightarrow \text{Left Child}, x_j > \tau \rightarrow \text{Right Child}$$

The final prediction at a leaf is the majority class of samples reaching that node:

$$\hat{y} = \arg \max_{k \in \{0,1\}} \sum_{i \in \text{leaf}} \mathbf{1}[y_i = k]$$

Splitting Criterion (Gini Impurity)

Node impurity is measured using Gini Impurity:

$$\text{Gini}(D) = 1 - \sum_k p_k^2$$

For binary classification:

$$\text{Gini}(D) = 2p(1 - p)$$

The optimal split minimizes weighted impurity:

$$(j^*, \tau^*) = \arg \min_{j, \tau} \left(\frac{|L|}{|D|} \cdot \text{Gini}(L) + \frac{|R|}{|D|} \cdot \text{Gini}(R) \right)$$

Optimization Strategy

Decision Trees use greedy recursive splitting, selecting at each node the feature-threshold pair that minimizes Gini impurity. The process is repeated until leaf nodes are formed.

Overfitting is controlled using:

- `max_depth = 15`
- `min_samples_split = 10`

These constraints limit tree complexity and improve generalization.

Model Selection

Performance across feature sets:

- Flatten: Accuracy = 98.99%, F1 = 95.55%

- PCA: Accuracy = 99.22%, F1 = 96.56%
- HOG: Accuracy = 98.80%, F1 = 94.72%

PCA was selected as the optimal representation, as it reduces redundancy and noise while preserving the most informative variance directions (50 components), resulting in cleaner splits and better generalization.

Overall, PCA + Decision Tree achieved the best balance of accuracy and F1-score.

4.0 Model Comparison Discussion

All five models were evaluated under the same preprocessing pipeline and compared using Accuracy, Precision, Recall, and F1-score.

KNN (k=1 + PCA) achieved the best overall performance with **99.49% accuracy and 0.9779 F1-score**, along with the highest recall (0.9947), making it the most effective model in terms of correctly detecting the positive class with minimal false negatives.

Decision Tree (PCA) followed with **99.22% accuracy and 0.9656 F1-score**, showing strong performance but slightly lower generalization than KNN. Logistic Regression (HOG) also performed competitively (**99.11% accuracy, 0.9609 F1-score**) but was less sensitive in comparison. SVM achieved high recall (0.9682) but a lower F1-score (0.9018) due to increased false positives. Naive Bayes, despite improvements with PCA, remained the weakest due to its independence assumption limiting representational capacity.

Overall, KNN with PCA is selected as the best model, providing the strongest balance between accuracy and recall. The results indicate that **PCA-enhanced feature spaces combined with instance-based learning yield the most effective performance for this binary classification task.**

5.0 Github Link

[AbdelrahmanAdel-1/MNIST-ML-ImageClassifiers: Binary & Multi-class Image Classification on MNIST using classical ML algorithms — built from scratch](#)